

```
In [109... # Ezinne Akabueze, CMOR 220, Spring 2026, Project 10: Cryptography and
# Project 10: Cryptography and Simple Ciphers.ipynb
# This coding program is tasked to use the the Metropolis Algorithm to
# Last modified: 4/29/26
```

```
In [110... import numpy as np
import matplotlib.pyplot as plt
import copy

# Defining all necessary functions
def downlow(text):
    #inputs: encoded text
    # function that returns number output from 0,1,...,26 for the text of
    newnums = np.zeros(len(text), dtype='int64')
    for i in range(len(text)):
        if text[i] == '`':
            newnums[i]=0
        elif text[i] != '`': # if not a `, the loop handles it like a
            newnums[i] = ord(text[i]) - 96 # since a is originally 97,
        else: # if neither, the loop does nothing
            pass

    return newnums #outputs: list of numbers

def downlowinv(numbers):
    #inputs: list of nums that is supposed to be an example of the encrypt
    # make an empty text str where we will add letters into, then return t
    text = [None] * len(numbers)
    for i in range(len(numbers)):
        if numbers[i]==0:
            text[i] = '`'
        elif text[i] != 0:
            text[i] = chr(int(numbers[i]+96)) # now were turning a=1 b
        else: # if neither, the loop does nothing
            pass
    textf = ''.join(text).replace('`,`', ' ') # gives us spaces instead o

    return textf #outputs: actual readable text

def decoder(text, maxiter):
    #inputs: list of concerted numbers from downlow, maxiter is the number
    # use Metropolis algorithm with maxiter iterations to compute the best
    y = np.random.permutation(27)
    for i in range(maxiter):
        ymaybe=copy.copy(y) # copying original y guess
        index=np.random.permutation(27)[:2]
        (ymaybe[index[0]], ymaybe[index[1]]) = (ymaybe[index[1]], ymay
        if loglike(text,ymaybe,prob) > loglike(text,y,prob):
            y=ymaybe
        elif np.exp(-loglike(text,y,prob)+loglike(text,ymaybe,prob)) >
```

```

        y = ymaybe
    else:
        pass

    return y #outputs: "cipher key" basically the vector that will dec

def loglike(text, guess, prob):
    #inputs: encoded list but as numbers from downlow, guess is out guess
    # use for loop to get row/columns, put them into the summation fucntio
    value = 0
    for k in range(len(text)-1):
        i = guess[text[k]] #rows
        j=guess[text[k+1]] #cols
        value = value + np.log(prob[i,j])
    return value #outputs: probability of the number based on probabil

```

```

In [111... # Part 1:
print("Part 1")

prob = np.load('letterprob.npy') #getting the matrix for letter probs
with open('encoded.txt', encoding='utf8') as f:
    for line in f:
        TEXT = line.strip('encodedtext.txt')
        TEXT = TEXT[0: -1]

# initialize the different givens
maxiter = int(10**4)
number =downlow(TEXT)

# find the best cipher key using decoder
y = decoder(number,maxiter)

# decode the text
decnums = np.zeros(len(number), dtype='int64')
for i in range(len(number)):
    decnums[i] = y[number[i]]

decodedtext = downlowinv(decnums)
print(str(decodedtext))

```

Part 1

ever since computers there have always been ghosts in the machine random segments of code that have grouped together to form unexpected protocols unanticipated these free radicals engender questions of free will creativity and even the nature of what we might call the soul why is it that when some robots are left in darkness they will seek out the light why is it that when robots are stored in an empty space they will group together rather than stand alone how do we explain this behavior random segments of code or is it something more when does a perceptual schematic become consciousness when does a difference engine become the search for truth when does a personality simulation become the bitter mote of a sou

